

PIMR++: Prime-Indexed Multi-Resolution Long-Context Module

Executive summary

PIMR++ is a **stable, auditable multiresolution attention framework**. The core claim is **not** “primes are special,” but that a **deterministic, well-spread span ladder** (prime-derived as one concrete construction) + **audited sparse gating** can **reduce hard training instabilities** and improve **worst-case OOD behavior** in long-context settings.

A *separate, quarantined* hypothesis tests whether an **Euler-motif (periodic) scoring term** provides an additional inductive bias beyond a capacity-matched null.

Core proposition (reframed)

PIMR++ designs a multiresolution mixture layer with:

1. **Deterministic span ladder**
2. Spans are strictly increasing and cover $[s_{\min}, s_{\max}]$ in log space.
3. “Prime-indexed” means primes provide a fixed low-discrepancy ordering into the ladder; performance should be distinguishable from composite-matched alternatives if primes matter.
4. **Audited gating**
5. Optional **Top-***** K** routing per token (or block) with explicit audits (usage histograms, entropy, saturation).
6. **Scientific controls**
7. Score families **Linear / Euler-motif / Null** are *capacity matched* (same basis count R , fixed amplitude β , same parameter budget).

Primary endpoints: **hard instability rate + OOD performance** (mean and tail/worst-quartile).

Components

A) Span ladder (`get_spans`)

Prime-normalized mapping on log-span space (strictly increasing, cost-aware coverage):

```
import numpy as np

def get_spans(m, s_min, s_max, get_first_n_primes):
    """Strictly increasing spans covering [s_min, s_max] on log scale.

    Args:
        m: number of spans
        s_min, s_max: min/max span (ints)
        get_first_n_primes: function returning first m primes as a list/np.array

    Returns:
        spans: list of strictly increasing ints
    """
    primes = np.asarray(get_first_n_primes(m), dtype=np.float64) # [2, 3,
5, ...]
    alpha = (primes - primes[0]) / (primes[-1] - primes[0]) # in [0,1]

    log_spans = np.log(s_min) + alpha * np.log(s_max / s_min)

    spans = []
    for ls in log_spans:
        s = int(np.ceil(np.exp(ls)))
        if spans:
            s = max(s, spans[-1] + 1)
        spans.append(s)

    return spans
```

Diagnostics to always log

- $u_k = \log s_k$ distribution across $[\log s_{\min}, \log s_{\max}]$
- distinct-span fraction (DSF), saturation, gating entropy

B) Operator family (example: pooled KV attention)

Let $X \in \mathbb{R}^{L \times d}$. For each span s_k , summarize keys/values via pooling:

- pooled length $M_k \approx \lceil L/s_k \rceil$

- compute cost roughly

$$\text{Cost} \propto L^2 \sum_{k=1}^m \frac{1}{s_k}$$

This makes ladder design directly comparable under fixed compute.

C) Gating (soft or Top-K, always audited)

For each token t , gating weights $g_{t,k}$ mix the per-span operators A_k :

$$Y_t = \sum_{k=1}^m g_{t,k} A_k(X)_t$$

Top-K routing option:

- compute logits $\ell_{t,k}$
- restrict support to Top- K
- renormalize with temperature τ

Audits:

- per-span usage U_k
 - entropy H_g
 - saturation rate (dominant span probability)
 - span-switch rate
-

D) Score families (quarantined)

Scores per span k :

$$a^{(k)}_{t,j} = \frac{q_t^{\text{top } k^{(k)}_{t,j}}}{\sqrt{d}} + \beta \phi^{(F)}(t-j; s_k)$$

where $F \in \{\text{Linear}, \text{Euler}, \text{Null}\}$.

Capacity matching requirements

- same basis count R
- fixed amplitude β across families
- same parameter budget

Euler motif example (conceptual): periodic bases using prime periods p_r . Null: same construction but replace p_r with seeded random integers (matched distribution).

E) Curvature regularization (stabilize gating)

Define second differences along span index k :

$$\mathcal{R}(\text{curv}) = \lambda \sum_t (g_{t,k+1} - 2g_{t,k} + g_{t,k-1})^2$$

Purpose: penalize thrashy span switching without forcing uniform usage.

Algorithm 1 — PIMR_Attention (pseudocode)

```
# Algorithm 1: PIMR_Attention
# X: (B, L, d_model)
# spans: [s_1, ..., s_m], strictly increasing
# F in {LINEAR, EULER, NULL}
# K_top: int or None (None => soft gating)
# tau: gating temperature
# beta: motif amplitude (fixed across families)
# lambda_curv: curvature reg coefficient

def PIMR_Attention(X, spans, F, K_top, tau, beta, lambda_curv, params):
    B, L, d = X.shape
    m = len(spans)

    # 0) Shared projections
    Q = X @ params.W_Q
    K_full = X @ params.W_K
    V_full = X @ params.W_V

    # 1) Per-span pooled K/V
    K_list, V_list, idx_map_list = [], [], []
    for s_k in spans:
        Kk, Vk, idx_map = P_pool(K_full, V_full, block_size=s_k)
        K_list.append(Kk)
        V_list.append(Vk)
        idx_map_list.append(idx_map)

    # 2) Gate logits over spans
    gate_logits = gate_net(X, spans, params)  # (B, L, m)

    # 3) Top-K or soft gating
    if K_top is not None:
        top_idx = topK_indices(gate_logits, K=K_top)  # (B, L, K_top)
        gate_mask = one_hot(top_idx, m).sum(axis=-2)  # (B, L, m)
        masked_logits = gate_logits + log(gate_mask + 1e-9)
        g = softmax(masked_logits / tau, axis=-1)
```

```

else:
    g = softmax(gate_logits / tau, axis=-1)

# 4) Per-span attention with controlled score families
A_list = []
for k in range(m):
    Kk, Vk = K_list[k], V_list[k]
    scores = (Q @ transpose(Kk)) / sqrt(Q.shape[-1]) # (B, L, M_k)

    if F == "LINEAR":
        motif = phi_linear(L, Kk.shape[1], spans[k], idx_map_list[k],
params)
    elif F == "EULER":
        motif = phi_euler(L, Kk.shape[1], spans[k], idx_map_list[k], params)
    elif F == "NULL":
        motif = phi_null(L, Kk.shape[1], spans[k], idx_map_list[k], params)
    else:
        motif = 0.0

    scores = scores + beta * motif
    attn = softmax(scores, axis=-1)
    Ak = attn @ Vk
    A_list.append(Ak)

# 5) Mixture
Y = 0.0
for k in range(m):
    Y = Y + g[..., k:k+1] * A_list[k]

Y = Y @ params.W_0

# 6) Curvature regularization
R_curv = 0.0
if lambda_curv > 0:
    dd = g[..., 2:] - 2*g[..., 1:-1] + g[..., :-2]
    R_curv = lambda_curv * (dd**2).mean()

# 7) Audits
audits = compute_audits(g, gate_logits, spans, K_top)

return Y, audits, R_curv

```

Diagnostic metrics (report per layer, aggregate across seeds)

Metric	Symbol	Definition	Healthy regime	Failure signature
Distinct-span fraction	DSF	$(\frac{1}{n} \sum_k \mathbb{E}_t[g_{t,k}])$	$\{k: \mathbb{E}_t[g_{t,k}] > \epsilon\}$	/m) (or Top-K variant) moderate, stable near 0 $\Rightarrow$ span collapse; near 1 + high entropy $\Rightarrow$ diffuse gating
Gating entropy	H_g	$\mathbb{E}_t[-\sum_k g_{t,k} \log(g_{t,k})]$	neither extreme	too low $\Rightarrow$ collapse; too high $\Rightarrow$ no specialization
Saturation rate	Sat	$\max_k \Pr_t[\arg \max_k g_{t,k}]$	bounded away from 1	$\rightarrow 1 \Rightarrow$ frozen dominant span
Span usage histogram	U_k	$\Pr_t[k \in \mathcal{K}_t]$ or $\mathbb{E}_t[g_{t,k}]$	spread across k	spikes on 1-2 spans; regime flips correlate w/ loss spikes
Coverage divergence	D_{cov}	KL/JS between binned $\log s_k$ usage and target	small	underused scales; often instability precursors
Curvature energy	E_{curv}	$\mathbb{E}_t \sum_k (\Delta^2 g_{t,k})^2$	decreases w/ reg	high $\Rightarrow$ thrashy switching
Span-switch rate	SSR	$\Pr_t[\arg \max g_t \neq \arg \max g_{t-1}]$	moderate	high $\Rightarrow$ unstable routing; low+high Sat $\Rightarrow$ frozen collapse
Compute-weighted usage	CWU	$\sum_k \mathbb{E}_t[g_{t,k}] c_k, c_k \propto 1/s_k$	stable vs budget	drift $\Rightarrow$ compute confound

Metric	Symbol	Definition	Healthy regime	Failure signature
Hard instability rate	HIR	fraction of runs with NaNs/ divergence/ unrecoverable spikes	low	primary endpoint

Validation protocol (fastest path)

Factorial core (3×2)

- Score family $F \in \{\text{Linear}, \text{Euler}, \text{Null}\}$
- Curvature regularization $\in \{\text{off}, \text{on}\}$

Required baselines

1. **Geometric multiscale ladder** (standard)
2. **Composite-matched ladder** (matches $\{s_k\}$ distribution / log-coverage)

Key falsifiers / stop rules

1. If no stability win vs geometric: **stop**.
2. If prime ladder $\nrightarrow$ composite-matched: publish as **audited deterministic multiresolution**.
3. If Euler $\nrightarrow$ Null (capacity matched): drop Euler and keep PIMR++ as engineering.

Cheap sanity checks

- **Prime permutation test:** keep $\{s_k\}$ fixed; permute mapping between span index and operator. No change $\Rightarrow$ primes are not doing extra work.

External references (background + anchoring comparisons)

Efficient / long-context attention

- Longformer: The Long-Document Transformer — Beltagy, Peters, Cohan (2020). arXiv:2004.05150.
- BigBird: Transformers for Longer Sequences — Zaheer et al. (NeurIPS 2020). arXiv:2007.14062.
- LongNet: Scaling Transformers to 1,000,000,000 Tokens — Ding et al. (2023). arXiv:2307.02486.
- Reformer: The Efficient Transformer — Kitaev, Kaiser, Levskaya (2020). arXiv:2001.04451.
- Linformer: Self-Attention with Linear Complexity — Wang et al. (2020). arXiv:2006.04768.
- Efficient Content-Based Sparse Attention with Routing Transformers — Roy et al. (TACL 2021). arXiv:2003.05997.

Mixture-of-Experts routing (audited sparse gating lineage)

- Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer — Shazeer et al. (2017). arXiv:1701.06538.
- Switch Transformers: Scaling to Trillion Parameter Models — Fedus, Zoph, Shazeer (2021; JMLR 2022). arXiv:2101.03961.

Survey (helpful for positioning)

- Efficient Transformers: A Survey — Tay et al. (2020). arXiv:2009.06732.

Optional link list (URLs placed in a code block)

```
https://arxiv.org/abs/2004.05150
https://arxiv.org/abs/2007.14062
https://arxiv.org/abs/2003.05997
https://arxiv.org/abs/2101.03961
https://arxiv.org/abs/1701.06538
https://arxiv.org/abs/2307.02486
https://arxiv.org/abs/2001.04451
https://arxiv.org/abs/2006.04768
https://arxiv.org/abs/2009.06732
```

Citizen Gardens © 2025 CC-NC-ND 4.0